

LayoverNow and Software Quality Assurance: A Review

1. What is LayoverNow?

- It is a Chrome extension (Manifest V3, JavaScript) that turns a normal flight connection into a free, planned stopover. The repo is on GitHub with instructor access, and it is built in Google Antigravity.
- Several airlines (Icelandair, TAP Portugal, Copa, Emirates) run free multi-day stopover programs, but they are hard to find and hard to compare against the extra travel time.
- The user enters an origin, destination, date, and how long they want to stop, and the tool surfaces routes whose layover cities are actually worth the detour.
- Target users are leisure and budget travelers, students, and anyone booking an international trip who wants more destinations for the same airfare.

2. What problem does it solve, and how?

- The core problem is decision support: is a given layover worth leaving the airport for, once you account for the extra detour. The tool does the comparison the traveler can't easily do by hand.
- Main features:
 - Offline airport search backed by a local geocoding database, so autocomplete is instant and has no network dependency. Typing a state code like "TX" filters to Texas airports.
 - A scoring engine that uses the Haversine great-circle formula to estimate detour overhead and rates each layover hub against known stopover programs.
 - Ranking by Best Value, Savings, or Minimum Detour, a flight-radar visualization, and one-click booking links for Skyscanner and Google Flights.
 - AeroAI Advisor, an AI layer that judges each layover for feasibility using length, season, and traveler interests, plus visa, currency, and weather context.

3. How it is built (architecture and versioning)

- V1 is a deterministic layover finder. It includes the airport search, the Haversine engine, static program scoring, the radar view, and booking links. It has no network dependency, so it can be tested in isolation, and it gets a full system test before any V2 work starts.
- V2 is the AI-assisted advisory layer. The AeroAI Advisor calls an external AI API for a reasoned, personalized assessment, and live enrichment adds visa, currency, and weather data.
- The version boundary is deliberate. All network and AI dependencies live in V2, so if a service fails, the failure stays out of the proven V1 core.

- The "smart" claim only applies to V2. V1 is rules-driven on purpose. V2 earns the label because it hands a real judgment to an external AI API and returns a recommendation, not a fixed score.

4. How LayoverNow connects to Software Quality Assurance

- Incremental development with isolation. Building and fully testing V1 before starting V2 is a clean version-control and change-control practice. It keeps deterministic logic separate from the parts that can fail unpredictably.
 - Two levels of testing. V1 testing is deterministic and repeatable (routing math, autocomplete, ranking, link generation, all offline). V2 testing is harder: it regression-tests every V1 path and then adds API latency, rate limits, malformed responses, and fallback to V1 when a service is down.
 - Fault tolerance and graceful degradation. The fallback to deterministic ranking plus response caching is a textbook reliability pattern for a system that depends on third-party services.
 - Risk management. Tracked risks include scope creep, Manifest V3 API changes, and the V2 dependency on outside AI and data APIs (cost, latency, outages), each with a stated mitigation.
 - The two quality areas that are hardest to assure here are the browser-extension surface and the AI layer, which is exactly where the two papers below come in.
-

5. Related IEEE Papers

5.1 Paper 1: Browser-extension quality and security

Citation and topic

- C. Obimbo, Y. Zhou, and R. Nguyen, "Analysis of Vulnerabilities of Web Browser Extensions," in 2018 International Conference on Computational Science and Computational Intelligence (CSCI), 2018, pp. 116 to 119. DOI: 10.1109/CSCI46756.2018.00029.
- It looks at how browser extensions, including Chrome ones, expose users to risk through the permissions they request.

Problem statement

- Most extensions are written by regular developers, not security experts, which makes them an easy target.

- The gap is that users grant permissions without understanding what an extension can actually do with them, and there is little checking of whether those permissions are abused.

Method and setup

- The authors did a security review of 50 Chrome extensions, 25 popular and 25 picked at random.
- They studied each one for vulnerabilities tied to privilege granting and privilege escalation, mostly by looking at the permissions in the manifest.

Results

- They found that the permission model leaves real room for abuse, since extensions can hold more access than they need and users can't tell the difference.
- They give recommendations for both developers and users to reduce the risk, centered on asking for fewer permissions.

Conclusion and link to LayoverNow

- The takeaway is to follow least privilege and keep the permission footprint small.
- This maps straight onto LayoverNow's quality story. It is a Manifest V3 extension, and keeping the core search offline already cuts its network exposure. The paper backs up treating the permission set and the V2 external calls as the main things to lock down and test.

5.2 Paper 2: Testing the AI layer

Citation and topic

- J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine Learning Testing: Survey, Landscapes and Horizons," IEEE Transactions on Software Engineering, vol. 48, no. 1, pp. 1 to 36, Jan. 2022. DOI: 10.1109/TSE.2019.2962027.
- A broad survey of how to test machine-learning and AI-based systems, which behave very differently from normal deterministic code.

Problem statement

- AI systems don't have a single correct output you can check against, so the usual "expected vs actual" testing doesn't fit. This is called the oracle problem.
- The paper pulls together the scattered research on how to test these systems for things like correctness, robustness, and fairness.

Method and setup

- It is a literature survey, not an experiment. The authors reviewed 144 papers on ML testing.
- They organize the field by what you test (correctness, robustness, fairness), what part you test (data, model, framework), and the workflow (generating tests, evaluating them).

Results

- The "results" are a structured map of techniques rather than numbers, including ideas like metamorphic testing, where you check that sensible changes to the input produce sensible changes to the output even when you can't name the one right answer.
- It also lays out open challenges and where the research is heading.

Conclusion and link to LayoverNow

- The main point is that AI components need their own testing approach, separate from deterministic code.
- That is exactly the V1 versus V2 split in LayoverNow. V1 can be tested with fixed expected values, but the AeroAI Advisor can't. This paper gives the vocabulary (oracle problem, robustness, metamorphic-style checks) for describing how V2 should be tested and why falling back to deterministic V1 is a sound safety net.

References

- C. Obimbo, Y. Zhou, and R. Nguyen, "Analysis of Vulnerabilities of Web Browser Extensions," in 2018 International Conference on Computational Science and Computational Intelligence (CSCI), 2018, pp. 116 to 119. DOI: 10.1109/CSCI46756.2018.00029.
- J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine Learning Testing: Survey, Landscapes and Horizons," IEEE Transactions on Software Engineering, vol. 48, no. 1, pp. 1 to 36, Jan. 2022. DOI: 10.1109/TSE.2019.2962027.